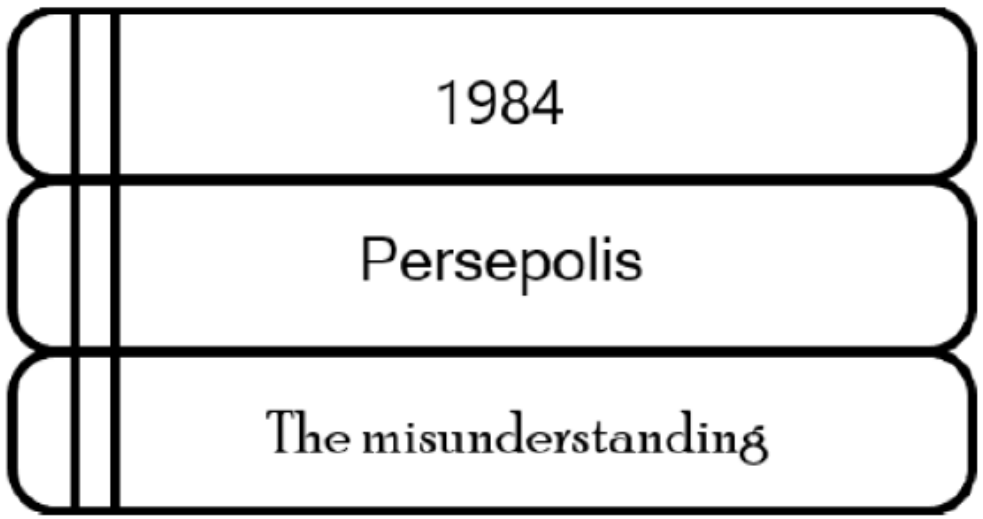
Data Structures and Algorithms

STACKS

Godofredo Avena

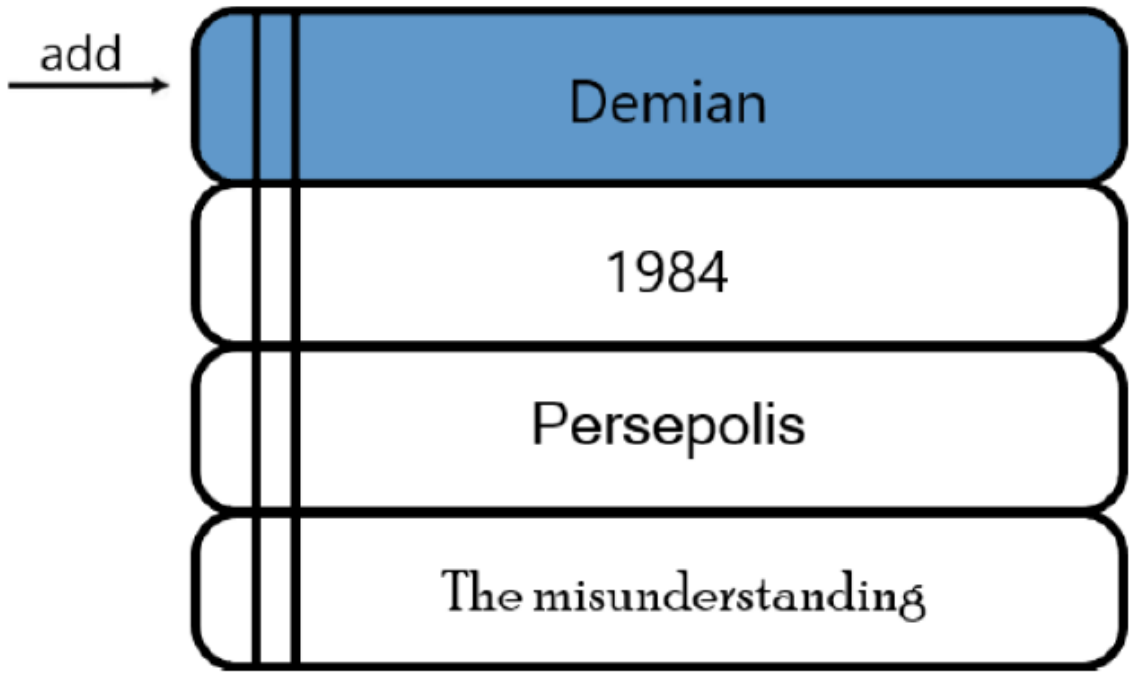
Stacks

- **LIFO:** Last-In First-Out
  - **Last inserted** item will be the **first** item to be **removed**



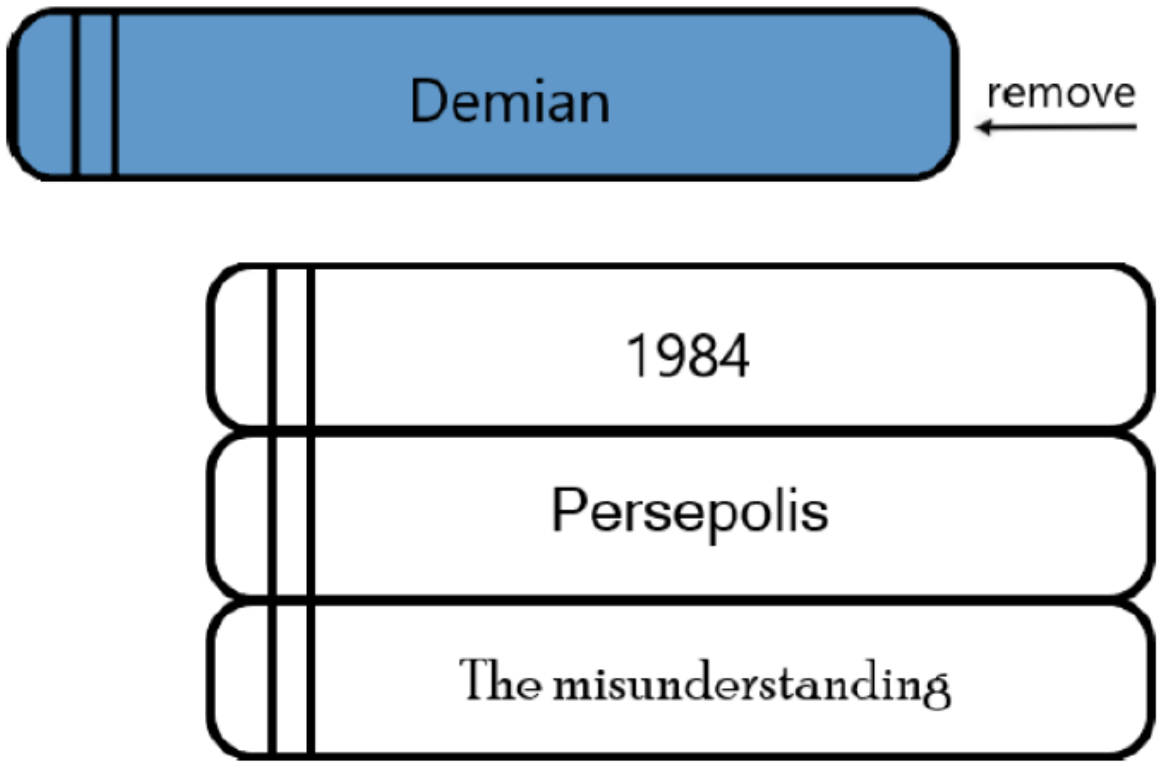
Stacks

- **LIFO:** Last-In First-Out
  - **Last inserted** item will be always the **first** item to be **removed**
- Can only **add** at the **top**
  - ***Pushing*** *onto the stack*



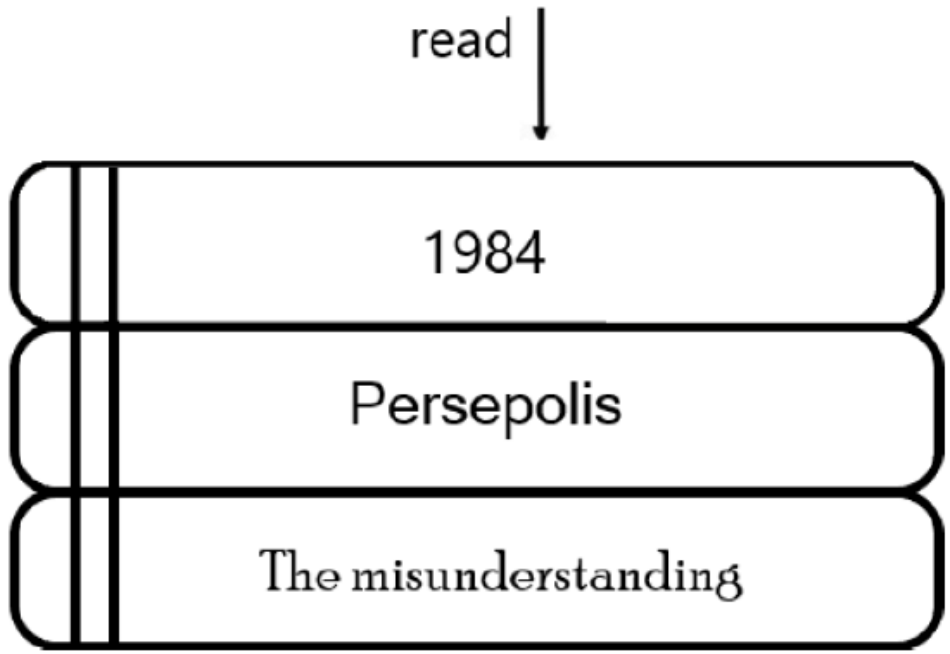
Stacks

- **LIFO:** Last-In First-Out
  - **Last inserted** item will be always the **first** item to be **removed**
- Can only **add** at the **top**
  - ***Pushing*** *onto the stack*
- Can only **take** from the **top**
  - ***Popping*** *from the stack*



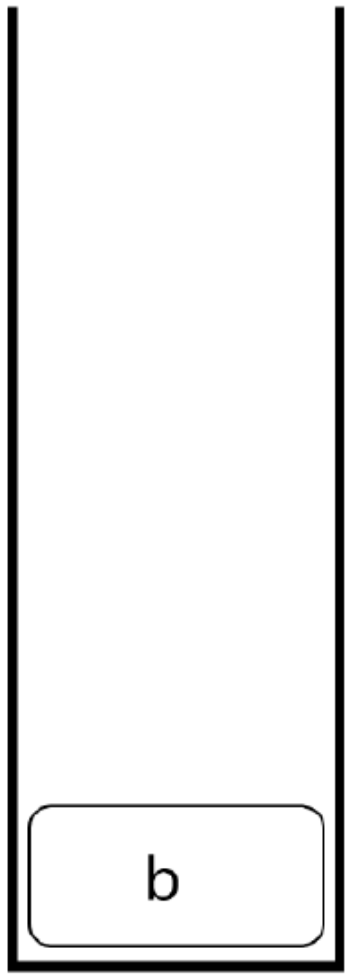
Stacks

- **LIFO:** Last-In First-Out
  - **Last inserted** item will be always the **first** item to be **removed**
- Can only **add** at the **top**
  - ***Pushing*** *onto the stack*
- Can only **remove** from the **top**
  - ***Popping*** *from the stack*
- Can only **read** the **last element**
  - ***Peeking*** *from the stack*



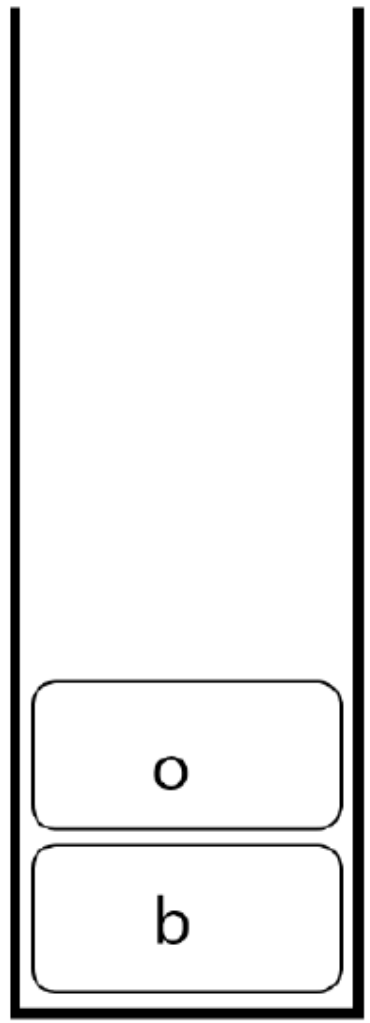
Stacks - real uses

- Undo functionality



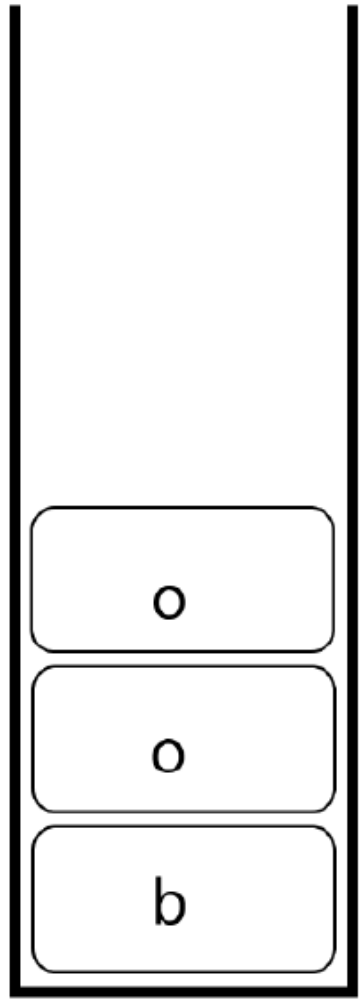
Stacks - real uses

- Undo functionality
  - **push** each keystroke



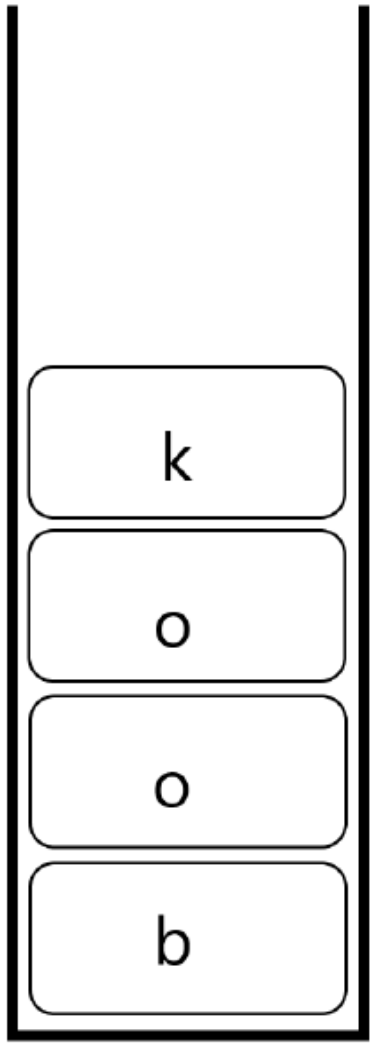
Stacks - real uses

- Undo functionality
  - **push** each keystroke



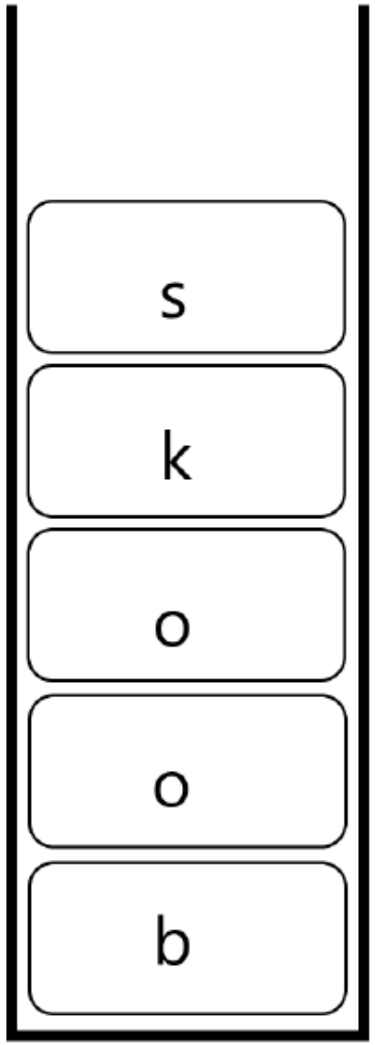
Stacks - real uses

- Undo functionality
  - **push** each keystroke



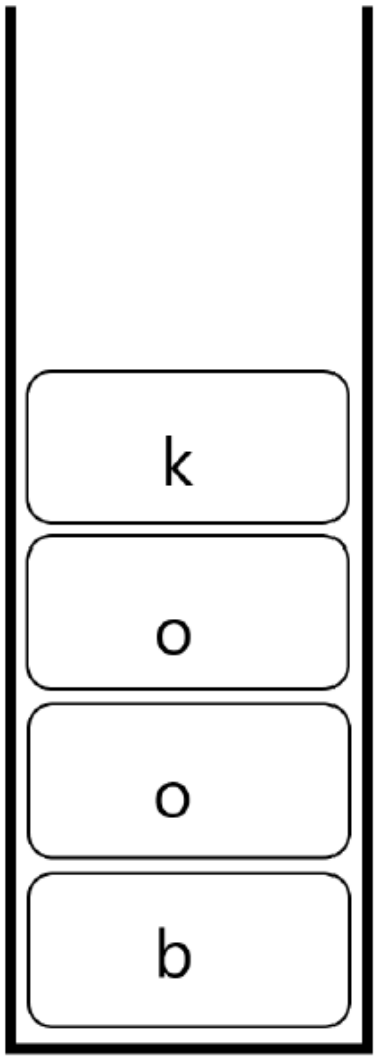
Stacks - real uses

- Undo functionality
  - **push** each keystroke



Stacks - real uses

- Undo functionality
  - **push** each keystroke
  - **pop** last inserted keystroke

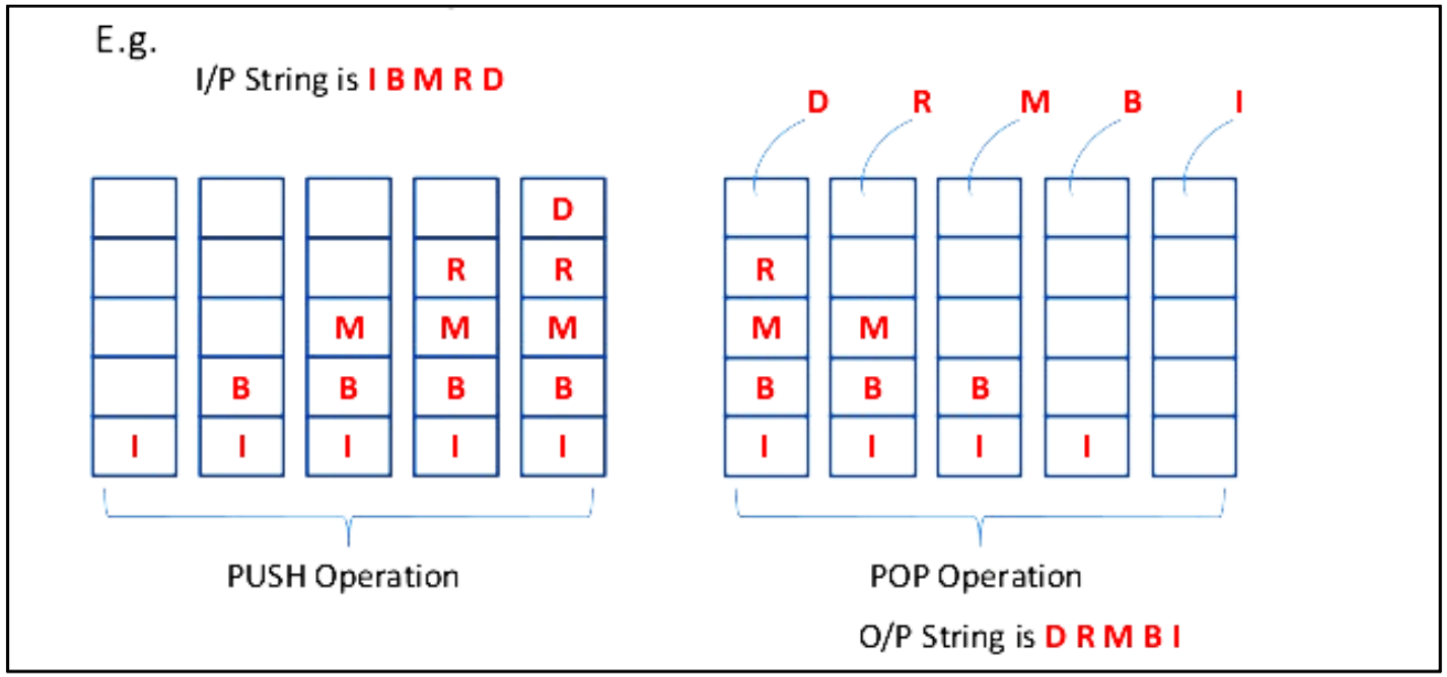


6.4 | REVERSING STRINGS

It is one of the simple applications of stack, where characters of string are pushed onto the stack one by one as the string is read from left to right. Once all characters are pushed, they are popped out. Since the last character pushed in is the first to be popped out. Subsequent pop operation results in reversal of strings.

Time Complexity: O (n) where n is the number of characters in stack.

Example 6.4.1.



Stack implementations : using Python array

```
class Stack:
    def __init__(self):
        self.stack = []

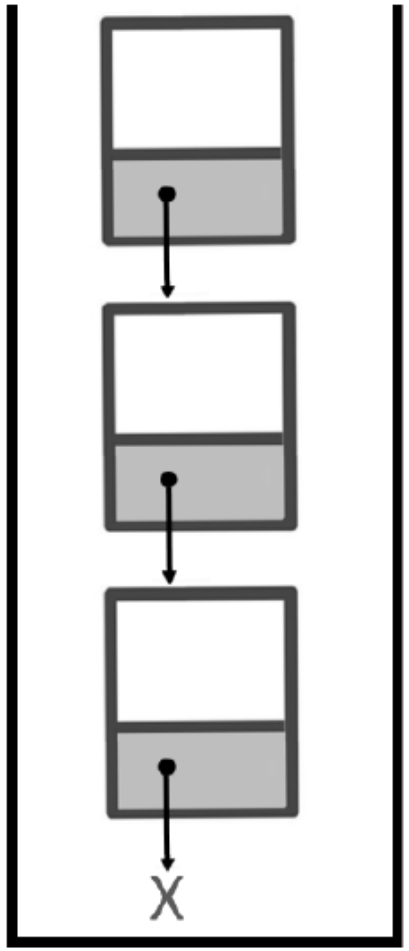
    def is_empty(self):
        return len(self.stack) == 0

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        else:
            raise IndexError("pop from empty stack")

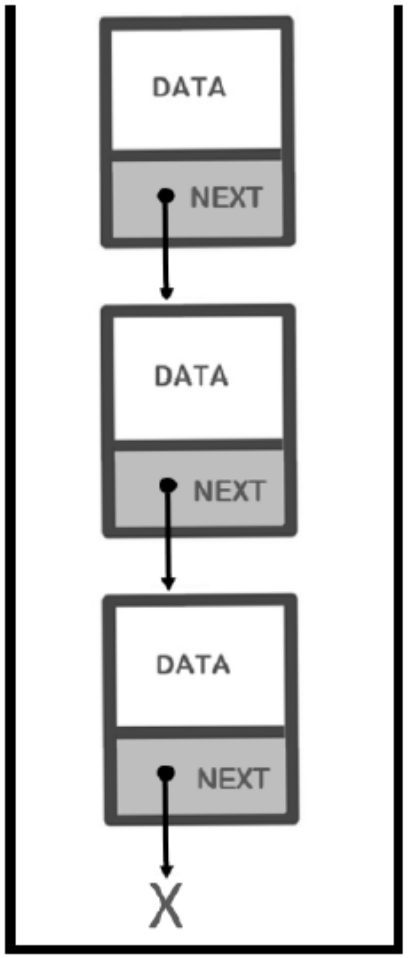
    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        else:
            raise IndexError("peek from empty stack")
```

Stacks - implementation using singly linked lists




```
class Node:
  def __init__(self,data):
    self.data = data
    self.next = None
```

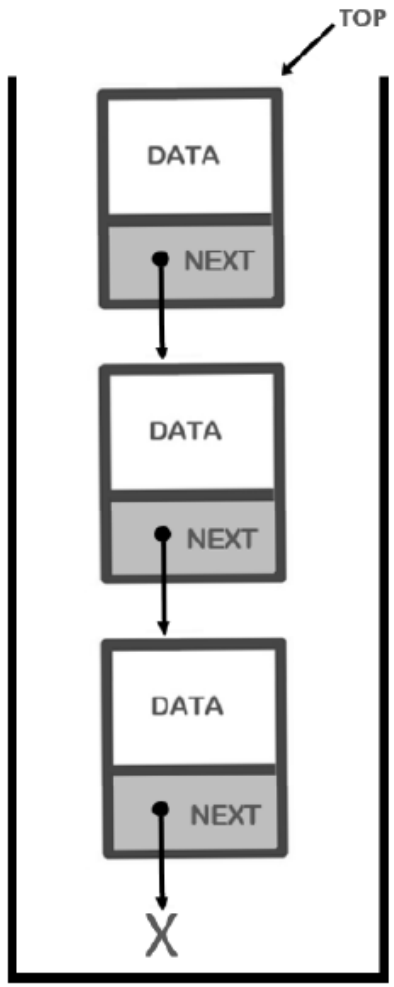
Stacks - implementation using singly linked lists




```
class Node:
  def __init__(self,data):
    self.data = data
    self.next = None
```

```
class Stack:
  def __init__(self):
    self.top = None
```

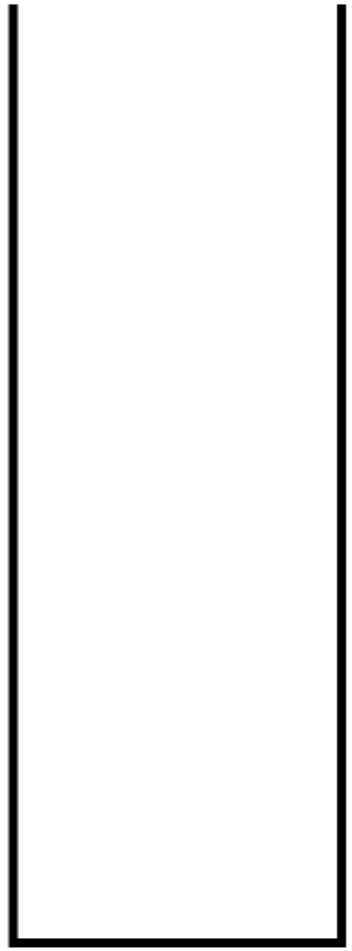
Stacks - implementation using singly linked lists



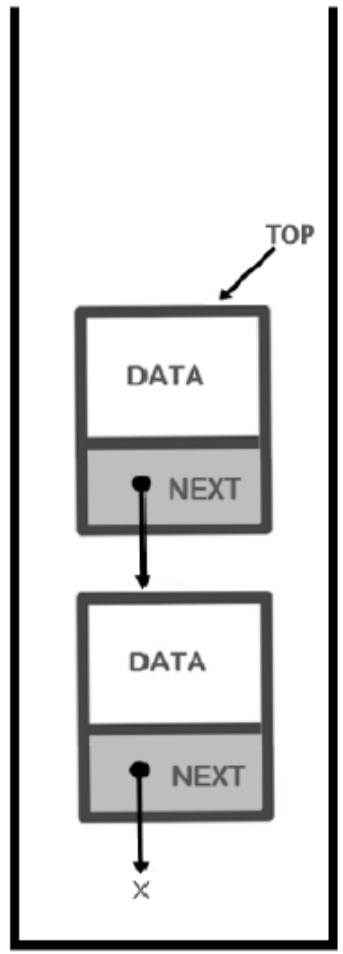

```
class Node:
  def __init__(self,data):
    self.data = data
    self.next = None
```

```
class Stack:
  def __init__(self):
    self.top = None
```

Stacks - push



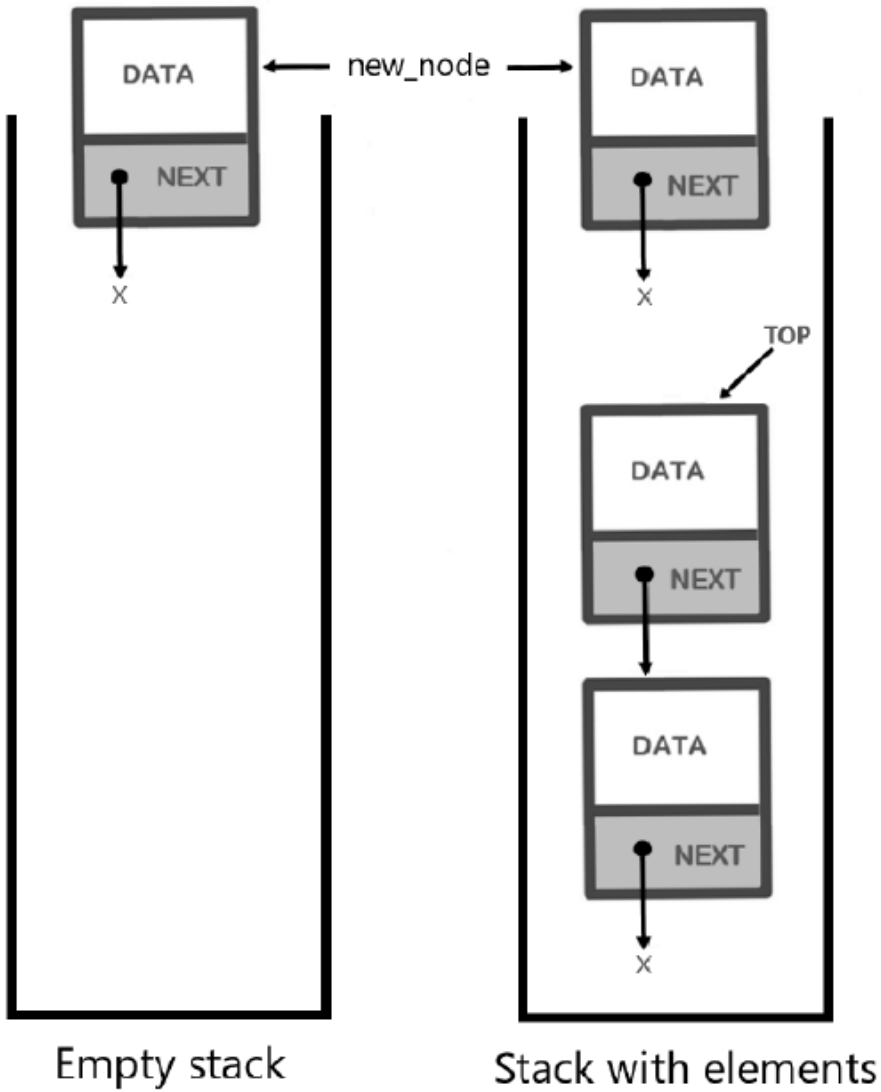
Empty stack



Stack with elements

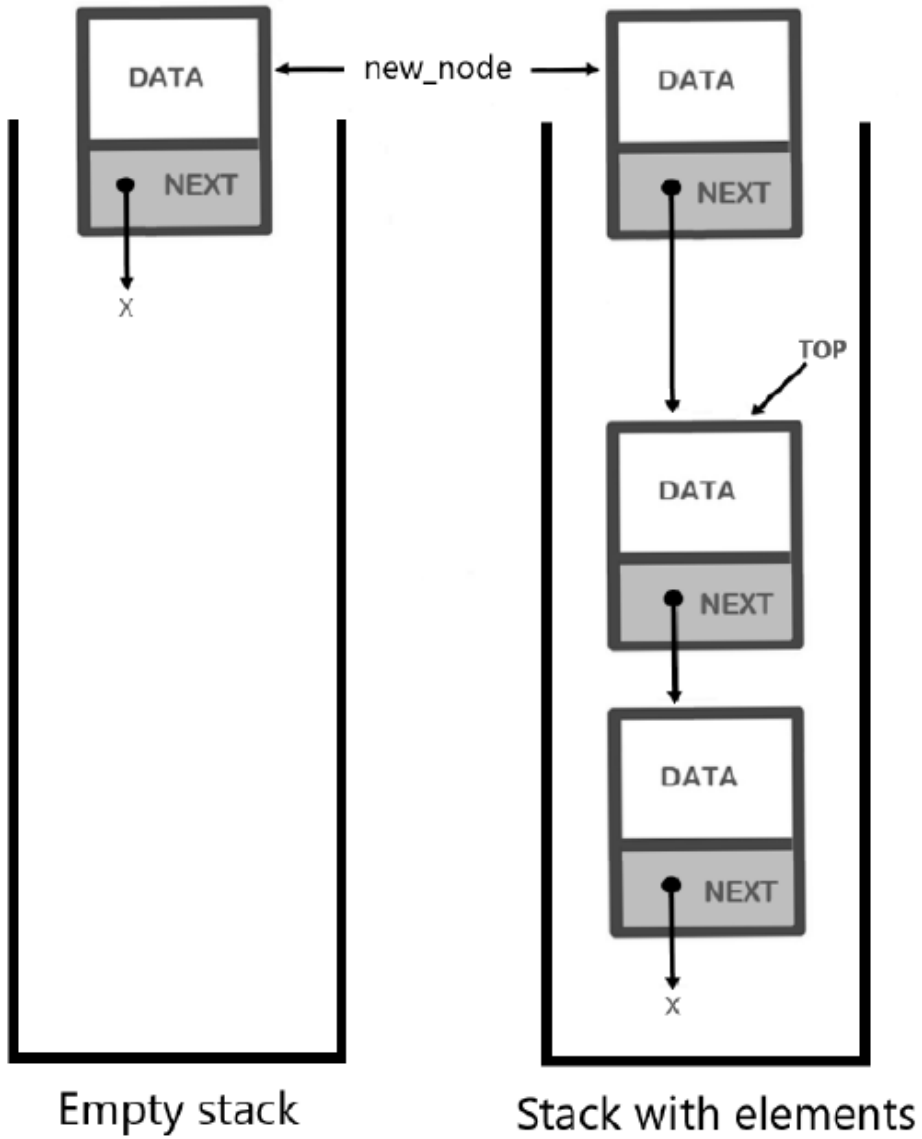
```
def push(self, data):
```

Stacks - push




```
def push(self, data):
  new_node = Node(data)
  if self.top:
```

Stacks - push

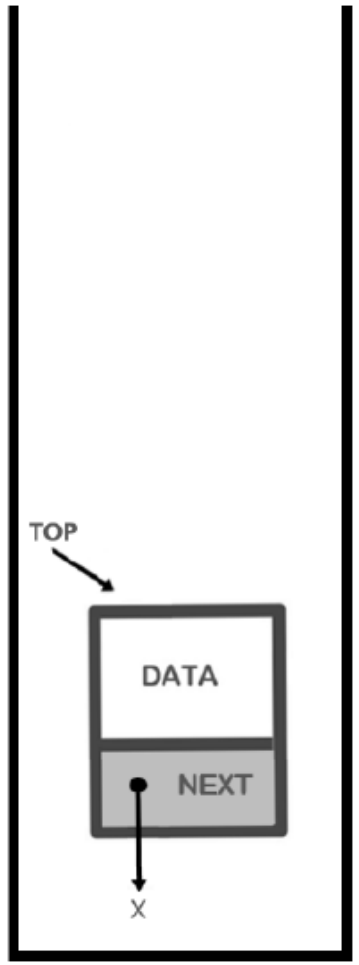


Empty stack

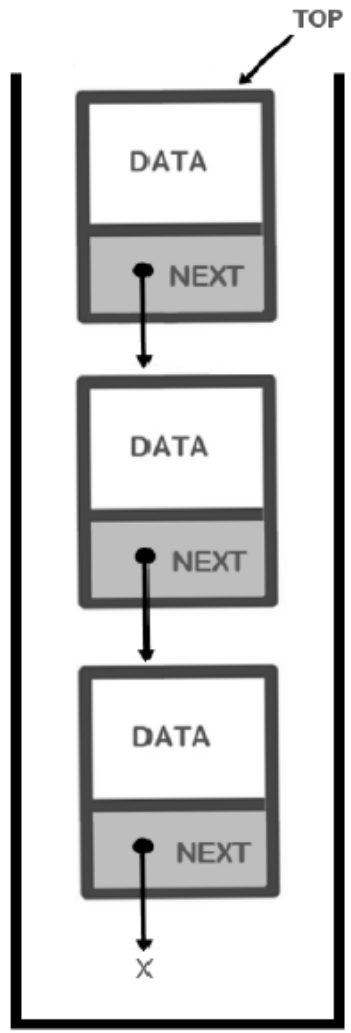
Stack with elements

```
def push(self, data):
  new_node = Node(data)
  if self.top:
    new_node.next = self.top
```

Stacks - push



Empty stack

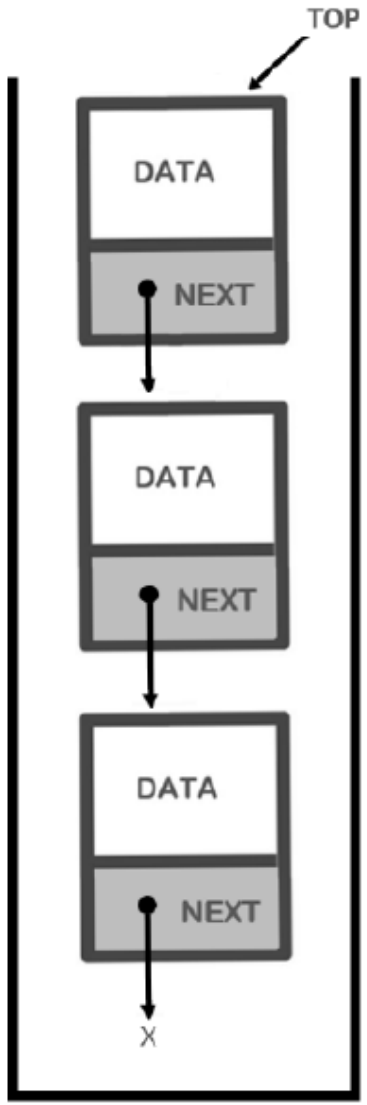


Stack with elements

```
def push(self, data):
  new_node = Node(data)
  if self.top:
    new_node.next = self.top
  self.top = new_node
```

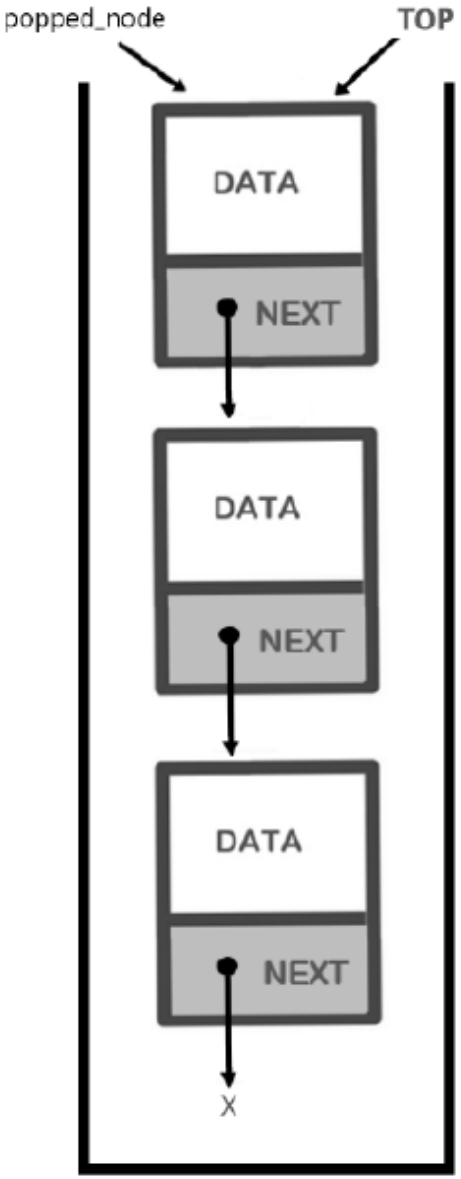
Stacks - pop

```
def pop(self):
  if self.top is None:
    return None
  else:
```



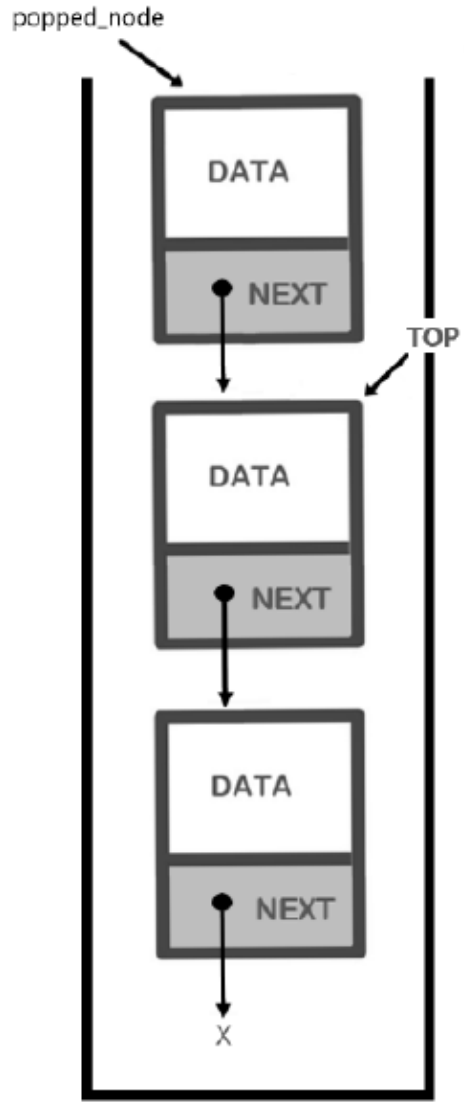
Stacks - pop

```
def pop(self):
  if self.top is None:
    return None
  else:
    popped_node = self.top
```



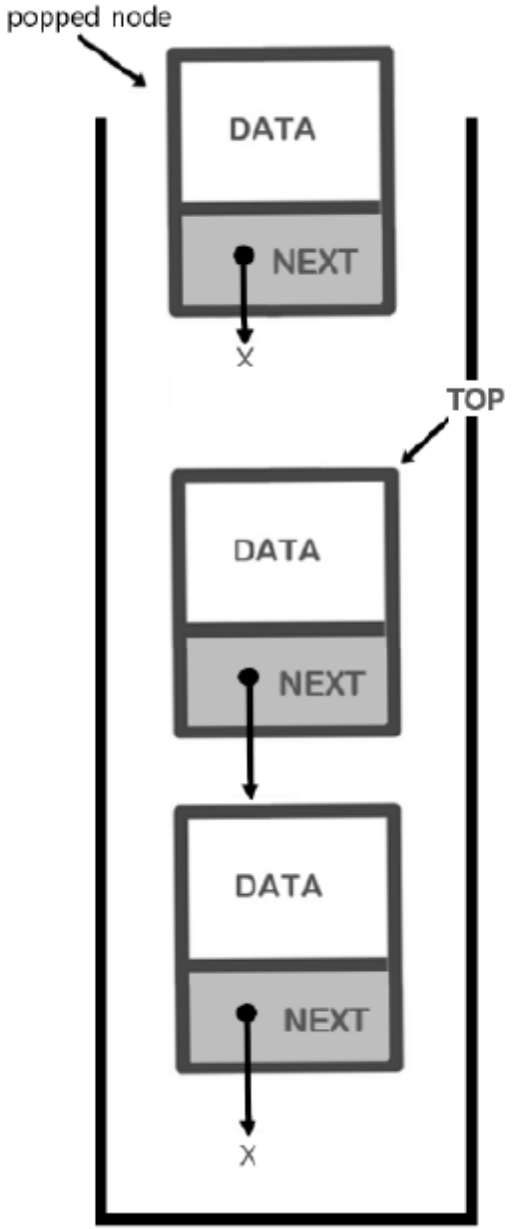
Stacks - pop

```
def pop(self):
  if self.top is None:
    return None
  else:
    popped_node = self.top
    self.top = self.top.next
```



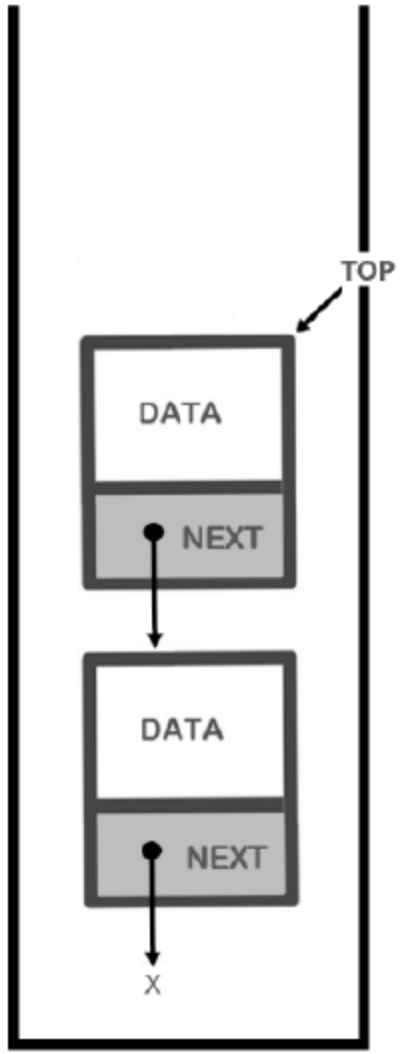
Stacks - pop

```
def pop(self):
  if self.top is None:
    return None
  else:
    popped_node = self.top
    self.top = self.top.next
    popped_node.next = None
```



Stacks - pop

```
def pop(self):
  if self.top is None:
    return None
  else:
    popped_node = self.top
    self.top = self.top.next
    popped_node.next = None
    return popped_node.data
```



Stacks - peek

```
def peek(self):
  if self.top:
    return self.top.data
  else:
    return None
```

LifoQueue in Python

- **LifoQueue:**
  - Python's **queue** module
  - behaves like a stack

```
import queue

my_book_stack = queue.LifoQueue(maxsize=0)
my_book_stack.put("The misunderstanding")
my_book_stack.put("Persepolis")
my_book_stack.put("1984")

print("The size is: ", my_book_stack.qsize())
```

```
The size is: 3
```

```
print(my_book_stack.get())
print(my_book_stack.get())
print(my_book_stack.get())
```

```
1984
Persepolis
The misunderstanding
```

```
print("Empty stack: ", my_book_stack.empty())
```

```
Empty stack: True
```

Stack Application: Conversion from Infix to Postfix

Infix notation, operators are written in-between their operands. This is the most common way of writing expressions that we use in everyday mathematics.

Example: For an addition of two numbers A and B, the infix notation would be **A + B**.

- Operator Precedence: For example, in the expression 3 + 4 * 5, MDAS rule applies unless parentheses are used to alter this order.
- Parentheses: Parentheses are often used to explicitly specify the order of operations.

Postfix notation, also known as Reverse Polish Notation (RPN), operators are written after their operands.
Example: For the same operation A + B, the postfix notation would be **A B +.**

- No Need for Parentheses:

Postfix notation is widely used in computer science because of its straightforward and parenthesis-free syntax

Stack Application: Conversion from Infix to Postfix

Precondition: An expression of the form <a> <operator> <b>

Postcondition: The algorithm should return the equivalent expression <a> <b> <operator>

a + b	a b +
a – b	a b -
a + b * c	a b c * +
(a + b) * (c + d)	a b + c d + *

Order of Precedence

Operator	Priority	Property	Example
^	3	Right Associative	a^b^c = a^(b^c)
* /	2	Left Associative	a/b/c = (a/b)/c
+ -	1	Left Associative	a – b – c = (a- b) - c

step-by-step explanation of how to convert an infix expression to postfix using a variation of the Shunting Yard algorithm:

1. Initialize Two Stacks:
    One for operators (let's call it ops).
    One for the output (let's call it output). Note: You can use a simple string variable for the output rather than stack
2. Scan the Infix Expression from Left to Right:
    If the Token is an Operand (number or variable):
        Add it to the output stack.
    If the Token is an Opening Bracket '(':
        Push it onto the ops stack.
    If the Token is a Closing Bracket ')':
        While the top of the ops stack is not an opening bracket:
            Pop from the ops stack and push onto the output stack.
        Pop the opening bracket from the ops stack (but don't add it to output).
    If the Token is an Operator (like +, -, *, /):
        While there's an operator at the top of the ops stack with greater or equal precedence:
            Pop operators from the ops stack onto the output stack.
        Push the current operator onto the ops stack.
3. After the Expression Has Been Fully Parsed:
    Pop any Remaining Operators from the ops Stack to the output Stack.
4. The output Stack Contains the Postfix Expression:
    The order of the tokens in the output stack is the postfix order.

Example:

Let's convert the infix expression

1. (A+B+C)*D to postfix:
2. A * (B + C) / D to postfix:
3. (A * (B +C^D)) to postfix

Example:
Let's convert the infix expression A * (B + C) / D to postfix:

1. A is an operand, output: A.
2. '*' is pushed to the ops stack.
3. '(' is pushed to the ops stack.
4. B is an operand, output: A B.
5. '+' is pushed to the ops stack.
6. C is an operand, output: A B C.
7.): pop until '(' from ops to output, output: A B C +. Pop the '(' from the ops stack.
8. '/' is an operator. It has lower precedence than '', so pop '' from ops to output, output: A B C + *, and then push '/' to ops.
9. D is an operand, output: A B C + * D.
10. End of expression, pop remaining '/' from ops to output.

Final Postfix Expression: A B C + * D /

6.5 | CONVERSION OF DECIMAL TO OTHER NUMBER SYSTEM

Here, the given decimal number is divided by the base (2 or 8 or 16) repeatedly and the corresponding elements are pushed onto the stack. Then the elements are popped out of the stack to give the required results.

Eg. 84 – 1010100 in binary

124 in octal

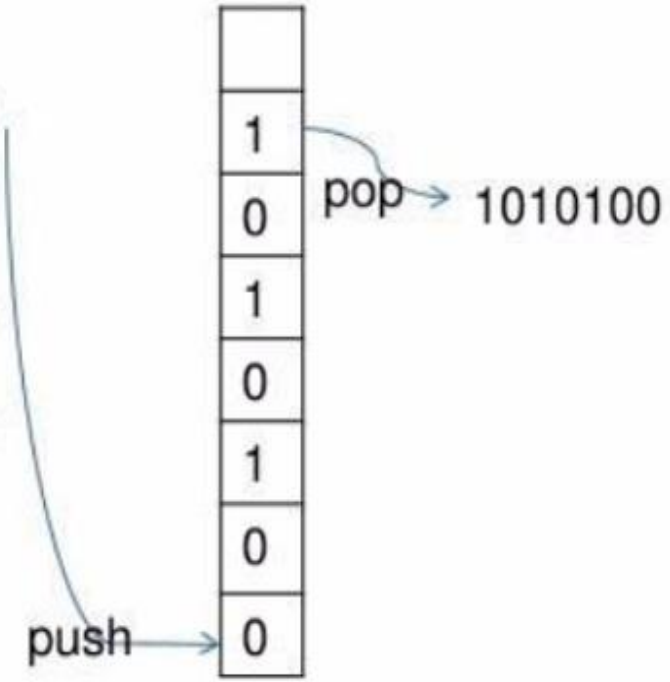
54 in hexadecimal

CONVERSION OF DECIMAL TO OTHER NUMBER SYSTEM

Base 2

84 / 2 = 42
42 / 2 = 21
21 / 2 = 10
10 / 2 = 5
5 / 2 = 2
2 / 2 = 1

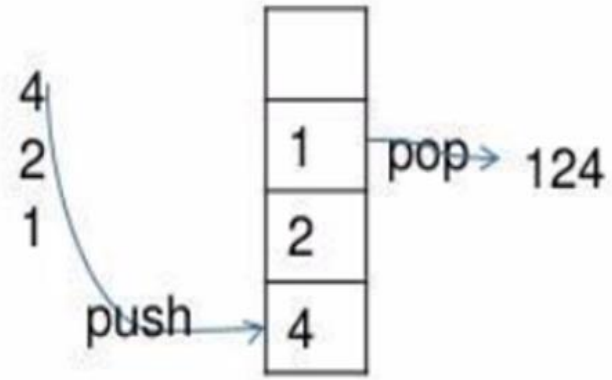
0 0 1 0 1 0 1



Base 8

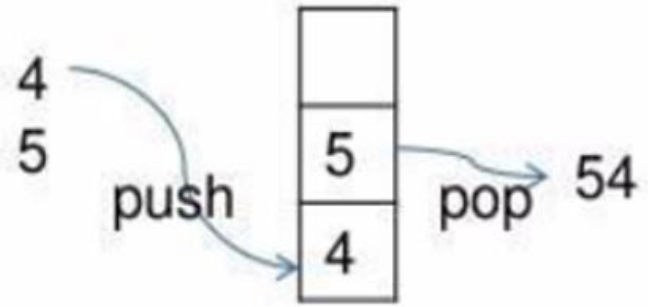
84 / 8 = 10
10 / 8 = 1

4 2 1



84 / 16 = 5

4 5



Base 16

6.4 | REVERSING STRINGS

It is one of the simple applications of stack, where characters of string are pushed onto the stack one by one as the string is read from left to right. Once all characters are pushed, they are popped out. Since the last character pushed in is the first to be popped out. Subsequent pop operation results in reversal of strings.

Time Complexity: O (n) where n is the number of characters in stack.

Example 6.4.1.

